claude-windows / 662b86ab-d2e3-4c02-8a0d-ec7a9071aba8

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\662b86ab-d2e3-4c02-8a0d-ec7a9071aba8\subagents\workflows\wf_498b24af-d16\agent-ad4c98b09f4111de8.jsonl`
- SHA-256: `b884e0cea45033daeff66bf54e4617132b59a232568894af063fecd78aacf7c0`
- Source modified: `2026-06-21T11:13:21+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_498b24af-d16`
- session_id: `662b86ab-d2e3-4c02-8a0d-ec7a9071aba8`

Transcript

1. user (2026-06-21T11:10:34.617Z)

Review the COMPUTE_DECOUPLED_SERVING M8 change just committed on branch feat/serving-m8-cost-ledger (HEAD). Inspect with: git diff origin/master...HEAD and read: backend/billing.py (pure cost math compute_cost_usd/apply_margin/to_inr); backend/infrastructure/database.py (_migrate_to_v6: cost columns on jobs + pricebook table seeded placeholder-v0 at 0.0; get_pricebook_rates, record_job_cost, get_job_cost, get_video_cost); backend/config/models.py (BillingConfig, default-OFF); backend/main.py (GET /api/jobs/{id}/cost + /api/videos/{id}/cost); backend/api/job_worker.py (_maybe_record_job_cost, gated); tests/test_billing.py + the cost tests in tests/test_api_endpoints.py. CONTRACT: DEFAULT-OFF ? billing.enabled=False records nothing (cost columns stay NULL = today); the seeded placeholder-v0 pricebook makes every cost 0; the v6 migration is additive-NULL + idempotent. USD is the source of truth, INR a display conversion. Report ONLY real, grounded issues with file:line. Empty findings if a lens finds nothing real.

LENS = MIGRATION & DEFAULT-OFF SAFETY. (1) Is _migrate_to_v6 truly additive-NULL + idempotent (re-runnable ALTERs via _add_column_idempotent; INSERT OR IGNORE on the unique pricebook key)? Does an interrupted upgrade (columns present, version stale) re-run cleanly? (2) Does anything change for billing.enabled=False (cost columns stay NULL, no served-path query reads them)? Is _maybe_record_job_cost a strict no-op when disabled, and best-effort (never raises into the job)? (3) The v6 bump from v5 ? are ALL schema-version pins updated across the test suite (grep for user_version==5)? (4) Does the new top-level billing import in database.py risk a circular import (database is imported widely; billing imports only typing)? (5) Route ordering for the two cost endpoints vs /api/jobs/{id} ? any FastAPI path shadowing? Find real safety/migration issues.

2. assistant (2026-06-21T11:10:37.263Z)

I'll review the M8 cost-ledger change through the MIGRATION & DEFAULT-OFF SAFETY lens. Let me start by pulling the latest and inspecting the diff.

3. user (2026-06-21T11:10:40.917Z)

```
feat/serving-m8-cost-ledger
582522b feat(serving): M8 ? per-job cost ledger + versioned pricebook + /cost API (default-OFF)
---DIFF STAT---
 backend/api/job_worker.py          |  27 +++++++
 backend/billing.py                 |  60 ++++++++++++++
 backend/config/models.py           |  16 ++++
 backend/infrastructure/database.py | 132 +++++++++++++++++++++++++++++++-
 backend/main.py                    |  28 +++++++
 tests/test_api_endpoints.py        |  46 ++++++++++
 tests/test_async_jobs.py           |   8 +-
 tests/test_billing.py              | 157 ++++++++++++++++++++++++++++++++++++
 tests/test_database.py             |  28 ++++---
 tests/test_job_worker_loop.py      |   8 +-
```

```
 10 files changed, 490 insertions(+), 20 deletions(-)
```

4. assistant (2026-06-21T11:10:42.839Z)

```
Let me read the full diff and the key files.
```

5. user (2026-06-21T11:10:44.019Z)

```
diff --git a/backend/infrastructure/database.py b/backend/infrastructure/database.py
index ba7a59c..6ae4b87 100644
--- a/backend/infrastructure/database.py
+++ b/backend/infrastructure/database.py
@@ -4,6 +4,8 @@ import logging
 from contextlib import contextmanager
 from typing import List, Dict, Any, Optional, Tuple

+from backend import billing  # pure cost math (no backend imports ? no cycle)
+
 logger = logging.getLogger(__name__)

 class Database:
@@ -102,7 +104,10 @@ class Database:

             if version < 5:
                 self._migrate_to_v5(cursor)
-            # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA user_version =
6
+
+            if version < 6:
+                self._migrate_to_v6(cursor)
+            # future: if version < 7: cursor.execute("ALTER TABLE ..."); PRAGMA user_version =
7

             conn.commit()

@@ -284,6 +289,53 @@ class Database:
         """)
         cursor.execute("PRAGMA user_version = 5")

+    def _migrate_to_v6(self, cursor):
+        # v6: per-job COST LEDGER (COMPUTE_DECOUPLED_SERVING M8, D8). NULLABLE cost columns on
+        # `jobs` (NULL = no cost recorded = byte-identical to today; nothing records until
+        # billing.enabled + a real pricebook) + a VERSIONED `pricebook` table. Cost is stored
in
+        # USD (matches GCP billing = one source of truth); INR is a display conversion at a
+        # configurable FX rate. ALTER (not recreate) so v1-v5 job rows survive; ADD COLUMNs are
+        # idempotent so an interrupted upgrade can re-run them. Additive ONLY ? no served
change.
+        for col, typ in (
+            ("owner_id", "TEXT"),               # who pays (the non-owner user / tenant)
+            ("compute_backend", "TEXT"),        # local_gpu | cloud_run | ...
+            ("gpu_type", "TEXT"),               # L4 | T4 | ... (selects the per-second GPU
rate)
+            ("gpu_active_seconds", "REAL"),
+            ("wall_seconds", "REAL"),
+            ("bytes_out", "INTEGER"),           # egress
+            ("gemini_cost_usd", "REAL"),        # provider-reported, already USD
+            ("pricebook_version", "TEXT"),
+            ("cost_usd", "REAL"),               # computed total (USD = source of truth)
+            ("cost_breakdown_json", "TEXT"),    # {gpu_seconds: .., egress_gb: .., gemini_usd:
..}
+        ):
+            self._add_column_idempotent(cursor, f"ALTER TABLE jobs ADD COLUMN {col} {typ}")
+        # Partial index keeps the NULL=no-cost fast path churn-free while making per-owner
billing cheap.
```

```
+        cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_owner ON jobs(owner_id) "
+                       "WHERE owner_id IS NOT NULL")
+        # Versioned pricebook: one row per (version, resource, gpu_type). Rates change + differ
by
+        # gpu_type ? re-version (never mutate a shipped version, so a recorded cost stays
auditable).
+        cursor.execute("""
+            CREATE TABLE IF NOT EXISTS pricebook (
+                version TEXT NOT NULL,
+                resource TEXT NOT NULL,
+                gpu_type TEXT NOT NULL DEFAULT '',
+                unit_price_usd REAL NOT NULL DEFAULT 0.0,
+                currency TEXT NOT NULL DEFAULT 'USD',
+                effective_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
+            )
+        """)
+        cursor.execute("CREATE UNIQUE INDEX IF NOT EXISTS idx_pricebook_key "
+                       "ON pricebook(version, resource, gpu_type)")
+        # Seed a PLACEHOLDER version with every rate 0.0 ? cost_usd computes to 0 (nothing
+        # user-facing changes). The owner INSERTs a real, calibrated version (real GCP $/GPU-s,
+        # egress $/GB) and flips billing.pricebook_version to it ? never overwrite this seed.
+        for resource, gpu_type in (("gpu_seconds", "L4"), ("gpu_seconds", "T4"),
+                                   ("wall_seconds", ""), ("egress_gb", "")):
+            cursor.execute(
+                "INSERT OR IGNORE INTO pricebook (version, resource, gpu_type, unit_price_usd,
"
+                "currency) VALUES ('placeholder-v0', ?, ?, 0.0, 'USD')", (resource, gpu_type))
+        cursor.execute("PRAGMA user_version = 6")
+
     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
         """Register or update a video row WITHOUT touching its child segments.

@@ -547,6 +599,84 @@ class Database:
             d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
             return d

+    # --- Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8) ------------------------------
+    def get_pricebook_rates(self, version: str, gpu_type: Optional[str] = None) -> Dict[str,
float]:
+        """Resolve a flat ``{resource: unit_price_usd}`` for a pricebook ``version``. The per-
second
+        GPU rate is selected by ``gpu_type``; non-GPU resources (wall/egress) carry
``gpu_type=''``.
+        An unknown version ? ``{}`` (so every cost computes to 0.0)."""
+        gt = gpu_type or ""
+        rates: Dict[str, float] = {}
+        with self._connect() as conn:
+            rows = conn.cursor().execute(
+                "SELECT resource, gpu_type, unit_price_usd FROM pricebook WHERE version = ?",
+                (version,)).fetchall()
+        for r in rows:
+            res, row_gt, price = r["resource"], (r["gpu_type"] or ""),
float(r["unit_price_usd"])
+            if res == billing.GPU_SECONDS:
+                if row_gt == gt:        # the GPU rate for THIS gpu_type
+                    rates[res] = price
+            else:
+                rates[res] = price      # wall/egress: gpu_type-independent
+        return rates
+
+    def record_job_cost(self, job_id: str, *, usage: Dict[str, float], pricebook_version: str,
+                        gpu_type: Optional[str] = None, compute_backend: Optional[str] = None,
+                        owner_id: Optional[str] = None, margin: float = 0.0) ->
Optional[Dict[str, Any]]:
```

```
+        """Compute the job's cost from ``usage`` × the pricebook + persist every cost column.
+        Returns the stored cost dict, or ``None`` on write failure. The caller gates this on
+        ``billing.enabled`` (default-OFF) ? nothing records until billing is turned on."""
+        rates = self.get_pricebook_rates(pricebook_version, gpu_type)
+        cost_usd, breakdown = billing.compute_cost_usd(usage, rates)
+        if margin:
+            cost_usd = billing.apply_margin(cost_usd, margin)
+        bytes_out = int(float(usage.get(billing.EGRESS_GB, 0.0) or 0.0) * 1e9)
+        ok = self._execute_write(
+            "UPDATE jobs SET owner_id=?, compute_backend=?, gpu_type=?, gpu_active_seconds=?, "
+            "wall_seconds=?, bytes_out=?, gemini_cost_usd=?, pricebook_version=?, cost_usd=?, "
+            "cost_breakdown_json=? WHERE id = ?",
+            (owner_id, compute_backend, gpu_type, usage.get(billing.GPU_SECONDS),
+             usage.get(billing.WALL_SECONDS), bytes_out, usage.get(billing.GEMINI_USD),
+             pricebook_version, cost_usd, json.dumps(breakdown), job_id),
+            f"Failed to record cost for job {job_id}")
+        if not ok:
+            return None
+        return {"cost_usd": cost_usd, "cost_breakdown": breakdown,
+                "pricebook_version": pricebook_version}
+
+    def get_job_cost(self, job_id: str) -> Optional[Dict[str, Any]]:
+        """The job's cost row (``None`` if the job doesn't exist; ``cost_usd`` is ``None``
until
+        recorded). ``cost_breakdown`` is the deserialized JSON breakdown."""
+        with self._connect() as conn:
+            row = conn.cursor().execute(
+                "SELECT id, video_id, owner_id, compute_backend, gpu_type, gpu_active_seconds,
"
+                "wall_seconds, bytes_out, gemini_cost_usd, pricebook_version, cost_usd, "
+                "cost_breakdown_json FROM jobs WHERE id = ?", (job_id,)).fetchone()
+        if not row:
+            return None
+        d = dict(row)
+        raw = d.pop("cost_breakdown_json", None)
+        d["cost_breakdown"] = json.loads(raw) if raw else {}
+        return d
+
+    def get_video_cost(self, video_id: str) -> Dict[str, Any]:
+        """Aggregate cost across a video's jobs: total ``cost_usd`` + the per-job rows. A video
is
+        1:1 with an infer job, but a re-ingest can produce several ? sum them."""
+        with self._connect() as conn:
+            rows = conn.cursor().execute(
+                "SELECT id, cost_usd, pricebook_version, cost_breakdown_json FROM jobs "
+                "WHERE video_id = ?", (video_id,)).fetchall()
+        jobs: List[Dict[str, Any]] = []
+        total = 0.0
+        for r in rows:
+            cost = r["cost_usd"]
+            if cost is not None:
+                total += float(cost)
+            jobs.append({
+                "job_id": r["id"], "cost_usd": cost, "pricebook_version":
r["pricebook_version"],
+                "cost_breakdown": json.loads(r["cost_breakdown_json"]) if
r["cost_breakdown_json"] else {},
+            })
+        return {"video_id": video_id, "total_cost_usd": round(total, 6), "jobs": jobs}
+
     # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2) -------------------------
     def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
         """Attach/replace a job's JSON ExecutionPolicy."""
```

6. user (2026-06-21T11:10:44.511Z)

```
diff --git a/backend/api/job_worker.py b/backend/api/job_worker.py
index 1385c7a..d3ad660 100644
--- a/backend/api/job_worker.py
+++ b/backend/api/job_worker.py
@@ -88,6 +88,7 @@ def _run_claimed_job(job: Dict[str, Any]) -> None:
         if result.get("video_id"):
             db_instance.set_job_video_id(job_id, result["video_id"])
         db_instance.mark_job_done(job_id, result)
+        _maybe_record_job_cost(job, result)  # M8 cost ledger (default-OFF)
     except _main.JobWaiting as w:
         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
@@ -97,6 +98,32 @@ def _run_claimed_job(job: Dict[str, Any]) -> None:
         db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")


+def _maybe_record_job_cost(job: Dict[str, Any], result: Dict[str, Any]) -> None:
+    """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
+    ``config.billing.enabled``. When on, compute the job's cost from the usage the pipeline
reported
+    (``result['usage']`` = {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the
+    versioned pricebook, and persist it. With the placeholder pricebook every rate is 0 ? cost
0,
+    so turning billing on changes no user-facing number until the owner inserts real rates.
+
+    Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job`` is the
+    claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
+    import backend.main as _main
+
+    billing_cfg = getattr(_main.global_config, "billing", None)
+    if not billing_cfg or not getattr(billing_cfg, "enabled", False):
+        return
+    try:
+        usage = (result or {}).get("usage") or {}
+        _main.db_instance.record_job_cost(
+            job["id"], usage=usage, pricebook_version=billing_cfg.pricebook_version,
+            gpu_type=usage.get("gpu_type"), compute_backend=(result or
{}).get("compute_backend"),
+            owner_id=job.get("owner_id"), margin=billing_cfg.margin,
+        )
+        logger.info("Recorded cost for job %s (pricebook=%s).", job["id"],
billing_cfg.pricebook_version)
+    except Exception as e:  # never wedge a completed job on bookkeeping
+        logger.warning("Cost recording for job %s failed (non-fatal): %s", job.get("id"), e)
+
+
 def _drain_due_jobs() -> int:
     """One worker tick: claim and run every currently-runnable job (queued, or waiting whose
     `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
diff --git a/backend/billing.py b/backend/billing.py
new file mode 100644
index 0000000..c7f3445
--- /dev/null
+++ b/backend/billing.py
@@ -0,0 +1,60 @@
+"""Per-job cost ledger ? pure cost math (COMPUTE_DECOUPLED_SERVING M8, decision D8).
+
+``cost_usd(job) = ?_resource usage(job, resource) × rate(resource, pricebook_version)``
+(the formula from the archived 05-COST-ATTRIBUTION design). Cost is computed + stored in
**USD**
+(matches GCP billing = one source of truth) and **displayed in INR** at a configurable FX rate
+(the Bangalore market). The pricebook is a versioned DB table
(``backend/infrastructure/database.py``);
+this module is the pure arithmetic over a resolved ``{resource: unit_price_usd}`` rates dict,
so it
```

```
+is unit-tested with zero IO and the DB layer owns the version/gpu_type lookup.
+
+**Default-OFF:** with the seeded ``placeholder-v0`` pricebook (every rate ``0.0``) every cost
is
+``0.0`` ? nothing user-facing changes until the owner inserts a real, calibrated pricebook
version.
+"""
+
+from __future__ import annotations
+
+from typing import Dict, Mapping, Tuple
+
+# Metered resource keys ? shared by the usage dict, the pricebook rows, and the breakdown.
+GPU_SECONDS = "gpu_seconds"     # GPU-active seconds (the big one) ? rate is per-second, per
gpu_type
+WALL_SECONDS = "wall_seconds"   # total wall time (separate from GPU so idle/sync isn't billed
as GPU)
+EGRESS_GB = "egress_gb"         # reel/byte egress in GB
+GEMINI_USD = "gemini_usd"       # provider-reported cost, ALREADY in USD (a passthrough, not
rate-×)
+
+# Resources whose cost = amount × unit_price (GEMINI_USD is excluded ? it is already a USD
amount).
+_RATE_MULTIPLIED = (GPU_SECONDS, WALL_SECONDS, EGRESS_GB)
+
+
+def compute_cost_usd(usage: Mapping[str, float],
+                     rates: Mapping[str, float]) -> Tuple[float, Dict[str, float]]:
+    """Return ``(cost_usd, breakdown)`` for one job.
+
+    ``usage`` maps a resource key to its measured amount (seconds / GB); ``rates`` maps a
resource
+    key to its USD unit price (already resolved for this pricebook version + gpu_type by the DB
+    layer). ``GEMINI_USD`` in ``usage`` is added verbatim (it is already a USD cost, not
multiplied).
+    Missing amounts/rates default to ``0.0`` ? with the placeholder pricebook the total is
``0.0``.
+    """
+    breakdown: Dict[str, float] = {}
+    total = 0.0
+    for res in _RATE_MULTIPLIED:
+        amount = float(usage.get(res, 0.0) or 0.0)
+        rate = float(rates.get(res, 0.0) or 0.0)
+        cost = amount * rate
+        if amount or rate:                # record the line even at $0 so the breakdown is
transparent
+            breakdown[res] = round(cost, 6)
+        total += cost
+    gemini = float(usage.get(GEMINI_USD, 0.0) or 0.0)
+    if gemini:
+        breakdown[GEMINI_USD] = round(gemini, 6)
+    total += gemini
+    return round(total, 6), breakdown
+
+
+def apply_margin(cost_usd: float, margin: float) -> float:
+    """Resale price = ``cost_usd × (1 + margin)``. ``margin`` is clamped ? 0 (never below
cost)."""
+    return round(cost_usd * (1.0 + max(0.0, float(margin))), 6)
+
+
+def to_inr(cost_usd: float, fx_inr_per_usd: float) -> float:
+    """Display conversion: USD ? INR at the configured FX rate (2 dp, the display currency)."""
+    return round(float(cost_usd) * float(fx_inr_per_usd), 2)
diff --git a/backend/config/models.py b/backend/config/models.py
```

```
index 92ad880..960f6c7 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -351,6 +351,21 @@ class DeploymentConfig(BaseModel):
     compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""


+class BillingConfig(BaseModel):
+    """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
+
+    **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay NULL =
+    today's behaviour). When enabled, a job's cost is computed + stored in **USD** via the
versioned
+    ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
+    version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the owner
+    inserts a real, calibrated version and points ``pricebook_version`` at it."""
+
+    enabled: bool = False
+    pricebook_version: str = "placeholder-v0"   # the seeded $0 version; owner flips to a real
one
+    fx_inr_per_usd: float = 83.0                  # USD?INR display rate (Bangalore market)
+    margin: float = 0.0                            # resale markup (?0); 0 = bill at cost
+
+
 class AppConfig(BaseModel):
     """Root configuration object.

@@ -372,6 +387,7 @@ class AppConfig(BaseModel):
     logging: LoggingConfig = Field(default_factory=LoggingConfig)
     storage: StorageConfig = Field(default_factory=StorageConfig)
     deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
+    billing: BillingConfig = Field(default_factory=BillingConfig)

     # Allow extra keys during transition so existing config.json files do not break.
     model_config = {
diff --git a/backend/main.py b/backend/main.py
index 6b89f6d..bc2ac10 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -39,6 +39,7 @@ from backend.config import (  # new typed config
 from backend.results import Failure  # standardized error/result contract
 from backend.reporting import emit_indexer_report
 from backend import storage  # M2: URI-aware input acquisition (local = identity passthrough)
+from backend import billing  # M8: per-job cost ledger (USD internal / INR display)

 app = FastAPI(title="Sports Highlight Indexer API")

@@ -83,6 +84,7 @@ from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay
addr
     _get_executor,
     _job_to_request,
     _MAX_DRAIN_WAKE_SEC,
+    _maybe_record_job_cost,
     _run_claimed_job,
     _schedule_next_drain_if_waiting,
 )
@@ -306,6 +308,32 @@ def process_video(req: ProcessRequest):
     )


+@app.get("/api/jobs/{job_id}/cost")
+def get_job_cost(job_id: str):
+    """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of truth
(matches
```

```
+    GCP billing); ``cost_inr`` is a display conversion at the configured FX rate. ``cost_usd``
is
+    null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
+    cost = db_instance.get_job_cost(job_id)
+    if cost is None:
+        raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
+    fx = global_config.billing.fx_inr_per_usd
+    usd = cost.get("cost_usd")
+    cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
+    cost["fx_inr_per_usd"] = fx
+    return cost
+
+
+@app.get("/api/videos/{video_id}/cost")
+def get_video_cost(video_id: str):
+    """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` + ``total_cost_inr`` +
the
+    per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce several."""
+    agg = db_instance.get_video_cost(video_id)
+    fx = global_config.billing.fx_inr_per_usd
+    agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
+    agg["fx_inr_per_usd"] = fx
+    return agg
+
+
 @app.get("/api/jobs/{job_id}")
 def get_job(job_id: str):
     """Job state: {status, progress, result, reports}. `status` = execution state
```

7. assistant (2026-06-21T11:10:47.318Z)

Now let me examine the migration ladder context, the route ordering, and the test suite for version pins. Let me look at the surrounding migration code and check route ordering.

8. user (2026-06-21T11:10:48.281Z)

```
40              try:
41                  conn.execute("PRAGMA journal_mode = WAL")
42              except sqlite3.OperationalError:
43                  pass  # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
44              return conn
45
46          def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
47              """Run ONE bool-returning single-statement write inside a connection scope.
48
49              Captures the connect/commit/except-logger.error/return-False boilerplate shared
by
50              the bool-returning single-statement writers. ``error_msg`` is the caller's exact
51              per-method message (already formatted with its own ids) ? logged verbatim on a
52              swallowed write fault so each writer keeps its specific log text. Returns True
on
53              success, False (after logging ``error_msg``) on any exception."""
54              try:
55                  with self._connect() as conn:
56                      conn.cursor().execute(sql, params)
57                      conn.commit()
58                  return True
59              except Exception as e:
60                  logger.error(f"{error_msg}: {e}")
61                  return False
62
63          @staticmethod
64          def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
65              """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.
66
```

```
            A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn:``
            transaction, so an interrupted v3/v4 block can leave the column added while
            ``user_version`` stays un-bumped ? and the next open would re-run the same
            ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
            future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
Swallow
            ONLY that specific error so the ladder stays crash-safe, matching the re-
runnable
            ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
            """
            try:
                cursor.execute(ddl)
            except sqlite3.OperationalError as exc:
                if "duplicate column name" not in str(exc).lower():
                    raise

        def _init_db(self):
            """Initializes tables for videos, play_segments, and events."""
            with self._connect() as conn:
                cursor = conn.cursor()

                self._create_base_tables(cursor)

                # Versioned migrations live here. PRAGMA user_version is the schema
                # contract ? bump it for every change and add an ordered `if version < N`
                # block below. Tests assert the expected version, so accidental drift fails
CI.
                version = cursor.execute("PRAGMA user_version").fetchone()[0]

                if version < 1:
                    self._migrate_to_v1(cursor)

                if version < 2:
                    self._migrate_to_v2(cursor)

                if version < 3:
                    self._migrate_to_v3(cursor)

                if version < 4:
                    self._migrate_to_v4(cursor)

                if version < 5:
                    self._migrate_to_v5(cursor)

                if version < 6:
                    self._migrate_to_v6(cursor)
                # future: if version < 7: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 7

                conn.commit()

        def _create_base_tables(self, cursor):
            """Create the videos / play_segments / events base tables (idempotent)."""
            # Videos Table
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS videos (
                    id TEXT PRIMARY KEY,
                    filepath TEXT NOT NULL,
                    processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    status TEXT NOT NULL,
                    validation_results TEXT
                )
            """)

            # Play Segments Table (Extensible metadata JSON)
```

```
128             cursor.execute("""
129                 CREATE TABLE IF NOT EXISTS play_segments (
130                     id INTEGER PRIMARY KEY AUTOINCREMENT,
131                     video_id TEXT NOT NULL,
132                     start_time REAL NOT NULL,
133                     end_time REAL NOT NULL,
134                     duration REAL NOT NULL,
135                     server TEXT,
136                     receiver TEXT,
137                     metadata TEXT,
138                     FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
139                 )
140             """)
141 
142             # Events Table
143             cursor.execute("""
144                 CREATE TABLE IF NOT EXISTS events (
145                     id INTEGER PRIMARY KEY AUTOINCREMENT,
146                     segment_id INTEGER NOT NULL,
147                     timestamp REAL NOT NULL,
148                     type TEXT NOT NULL,
149                     player TEXT,
150                     details TEXT,
151                     FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
152                 )
153             """)
154 
155         def _migrate_to_v1(self, cursor):
156             # v1: raw candidate detections (pre-confirmation), detector-agnostic.
157             # Common fields are columns; detector-specific metrics go in `stats` JSON,
158             # so a new segmenter reuses this table by setting its own `detector`/`stats`.
159             cursor.execute("""
160                 CREATE TABLE IF NOT EXISTS candidate_segments (
161                     id INTEGER PRIMARY KEY AUTOINCREMENT,
162                     video_id TEXT NOT NULL,
163                     detector TEXT NOT NULL,
164                     chunk_index INTEGER,
165                     start_time REAL NOT NULL,
166                     end_time REAL NOT NULL,
167                     duration REAL NOT NULL,
168                     confidence REAL,
169                     stats TEXT,
```

9. user (2026-06-21T11:10:49.514Z)

```
64:    def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
219:        self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
220:        self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
TIMESTAMP")
232:        self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id TEXT")
233:        self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
user_id TEXT")
311:            self._add_column_idempotent(cursor, f"ALTER TABLE jobs ADD COLUMN {col} {typ}")
```

10. assistant (2026-06-21T11:10:51.372Z)

Now let me check the route ordering concern and grep for version pins across the test suite.

11. user (2026-06-21T11:10:53.457Z)

```
86:    version = conn.execute("PRAGMA user_version").fetchone()[0]
89:    assert version == 6
108:    version = conn.execute("PRAGMA user_version").fetchone()[0]
114:    assert version == 6      # v6 = per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8)
```

```
229:    conn.execute("PRAGMA user_version = 3")
237:    assert conn.execute("PRAGMA user_version").fetchone()[0] == 6
255:    on its own, so a crash after the ADD COLUMNs but before the ``PRAGMA user_version``
263:    # but the process died before the version bump, so user_version is stuck at 2.
266:    conn.execute("PRAGMA user_version = 2")
275:    assert conn.execute("PRAGMA user_version").fetchone()[0] == 6
338:    assert m["n_videos_at_fit"] == 6
351:def test_upsert_is_idempotent_on_user_version(tmp_path):
356:        fusion_config={"min_crossings": 5},
362:    assert m["fusion_config"] == {"min_crossings": 5}
466:        ok = db.add_event(1, 5.0, "smash")
503:        ok = db.set_job_waiting("job-3", 5.0)
```

12. user (2026-06-21T11:10:53.994Z)

```
CLAUDE.md:45:  user_version` migration ladder in `backend/infrastructure/database.py`.
docs\ANALYZERS_AND_RECONCILERS.md:328:  `PRAGMA user_version` migration ladder
(`database.py:88`, `if version < 3:`) so applied corrections are
docs\ANALYZERS_AND_RECONCILERS.md:522:  (`user_version < 3`); apply behind `reconcile.apply`;
**rollback = flip the flag.**
docs\CODE_MAP.md:216:[Omitted long matching line]
docs\CODE_MAP.md:335:| Add a DB column/table |
`@backend/infrastructure/database.py::Database._init_db` migration ladder + bump `PRAGMA
user_version` (else version-assert tests fail) |
docs\CODE_MAP.md:374:| `test_database.py` | `infrastructure.database.Database`
schema/CRUD/migrations/`user_version`/`EXPECTED_CANDIDATE_COLUMNS` guard |
docs\EXECUTION_POLICY.md:89:  (user_version 3), `claim_due_job` (atomic, earliest-due, compare-
and-set), `set_job_waiting`,
docs\PLATFORM_ARCHITECTURE.md:64:Grounding: `backend/infrastructure/database.py` (schema +
`PRAGMA user_version`
docs\PLATFORM_ARCHITECTURE.md:221:graduate the `PRAGMA user_version` migration discipline to
**Alembic** under Postgres.
docs\PLATFORM_ARCHITECTURE.md:475:| **P0** | ? **DONE ? Async job model, single-tenant**
(DEFERRED #16) | `jobs` table via `user_version` ladder; single-worker `ThreadPoolExecutor`;
`/api/process` ? 202 + job_id; `GET /api/jobs/{id}` status |
docs\archives\DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md:94:> `jobs` table at
`user_version` 2, ThreadPoolExecutor(1), startup sweep for orphaned jobs, static UI
docs\archives\DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md:108:
(`backend/infrastructure/database.py::_init_db`, bump `PRAGMA user_version` 1 ? 2,
docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md:65:  even if unpopulated. ? This is a real DB delta (a
`PRAGMA user_version` migration): `play_segments` today has
docs\PERSONALIZATION_PLAN.md:83:**Per-user data model** (migrations follow the `PRAGMA
user_version` ladder; current = v5):
docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md:55:   Canonical JSONL per rally
(video_id MD5, rally_ordinal, sport, start/end, shot_count, ending_reason/fault_tags,
label_type, source, consent/training_opt_in, split=BY MATCH, trajectory_ref, ...). Wire to
durable per-video **event store** (the conversational-QA seam: "extract once, query many"). DB
delta: extend `play_segments`, new `highlights`/events table + `PRAGMA user_version` migration.
Reserve fault/event taxonomy now (service_fault etc.). No Gemini-sourced rows in training views.
docs\UI_ALPHA_EXECUTION_PLAN.md:30:- [x] **PR-1 · B1 async job model (#16):** `jobs` table
(user_version 1?2), `POST /api/process` ? 202 + `job_id`, `GET /api/jobs/{job_id}`,
ThreadPoolExecutor worker, restart/crash state handling ? **MERGED 2026-06-11, PR #87** (CI
green: 333 passed; machine-side real-video checks fold into L3)
docs\UI_PROPOSAL.md:60:| B1 | **Async job model ? DEFERRED_HARDENING_PLAN #16** | The committed
next PLATFORM slice; `POST /api/process` ? 202 + `job_id`, `jobs` table (user_version 1?2),
ThreadPoolExecutor, `GET /api/jobs/{id}`. **Approving this proposal = greenlighting #16.** A
5?60 min blocking HTTP call cannot sit behind a tunnel UI. |
backend\infrastructure\database.py:69:        ``user_version`` stays un-bumped ? and the next
open would re-run the same
backend\infrastructure\database.py:88:            # Versioned migrations live here. PRAGMA
user_version is the schema
backend\infrastructure\database.py:91:            version = cursor.execute("PRAGMA
user_version").fetchone()[0]
backend\infrastructure\database.py:110:             # future: if version < 7:
cursor.execute("ALTER TABLE ..."); PRAGMA user_version = 7
```

```
backend\infrastructure\database.py:183:        cursor.execute("PRAGMA user_version = 1")
backend\infrastructure\database.py:210:        cursor.execute("PRAGMA user_version = 2")
backend\infrastructure\database.py:222:        cursor.execute("PRAGMA user_version = 3")
backend\infrastructure\database.py:242:        cursor.execute("PRAGMA user_version = 4")
backend\infrastructure\database.py:280:            CREATE UNIQUE INDEX IF NOT EXISTS
idx_user_models_user_version
backend\infrastructure\database.py:290:        cursor.execute("PRAGMA user_version = 5")
backend\infrastructure\database.py:337:        cursor.execute("PRAGMA user_version = 6")
docs\archives\past_projects\DECOUPLED_COMPUTE\05-COST-ATTRIBUTION.md:38:Add via the existing
`_add_column_idempotent` / `PRAGMA user_version` migration ladder:
tests\test_billing.py:39:        ver = conn.execute("PRAGMA user_version").fetchone()[0]
tests\test_async_jobs.py:10:- migration: fresh DB lands at user_version 2 with a jobs table; v1
DBs upgrade in place
tests\test_async_jobs.py:83:        assert conn.execute("PRAGMA user_version").fetchone()[0] ==
6
tests\test_async_jobs.py:101:    conn.execute("PRAGMA user_version = 1")
tests\test_async_jobs.py:107:        assert c.execute("PRAGMA user_version").fetchone()[0] == 6
tests\test_database.py:86:    version = conn.execute("PRAGMA user_version").fetchone()[0]
tests\test_database.py:108:    version = conn.execute("PRAGMA user_version").fetchone()[0]
tests\test_database.py:229:    conn.execute("PRAGMA user_version = 3")
tests\test_database.py:237:    assert conn.execute("PRAGMA user_version").fetchone()[0] == 6
tests\test_database.py:255:    on its own, so a crash after the ADD COLUMNs but before the
``PRAGMA user_version``
tests\test_database.py:263:    # but the process died before the version bump, so user_version
is stuck at 2.
tests\test_database.py:266:    conn.execute("PRAGMA user_version = 2")
tests\test_database.py:275:    assert conn.execute("PRAGMA user_version").fetchone()[0] == 6
tests\test_database.py:351:def test_upsert_is_idempotent_on_user_version(tmp_path):
tests\test_job_worker_loop.py:275:        assert conn.execute("PRAGMA
user_version").fetchone()[0] == 6
tests\test_job_worker_loop.py:279:    # A pre-migration legacy DB (only `videos`, user_version
0) must run the FULL ordered
tests\test_job_worker_loop.py:287:    conn.execute("PRAGMA user_version = 0")
tests\test_job_worker_loop.py:292:        assert c.execute("PRAGMA user_version").fetchone()[0]
== 6
```

13. assistant (2026-06-21T11:10:57.599Z)

All version pins are updated to 6. Now let me verify the FastAPI route ordering concern. The two cost endpoints are defined before `/api/jobs/{job_id}`. Let me check the exact ordering and whether `/api/jobs/{job_id}/cost` could shadow or be shadowed.

14. user (2026-06-21T11:10:59.161Z)

```
61:@app.get("/", response_class=HTMLResponse)
131:@app.get("/api/config")
135:@app.get("/api/health")
268:@app.post("/api/process", status_code=202)
311:@app.get("/api/jobs/{job_id}/cost")
326:@app.get("/api/videos/{video_id}/cost")
337:@app.get("/api/jobs/{job_id}")
370:@app.get("/api/highlights/{video_id}/{filename}")
394:@app.post("/api/query")
405:@app.post("/api/compile")
```

15. assistant (2026-06-21T11:11:02.814Z)

Route ordering is correct ? `/api/jobs/{job_id}/cost` (line 311) is registered before `/api/jobs/{job_id}` (line 337). FastAPI matches in registration order, and even though `/cost` is more specific, registering it first guarantees no shadowing. Good.

Now let me verify the circular import claim and check `_connect` and `_execute_write` semantics for the new methods. Let me also look at the test files for billing and the migration tests to confirm the idempotency tests exist.

16. user (2026-06-21T11:11:03.758Z)

```
244         assert "cost_usd" in _table_cols(db_path, "jobs")  # v6 cost columns back-filled
(NULL)
245         assert "user_id" in _table_cols(db_path, "candidate_segments")
246
247         v = db2.get_video("vidA")
248         assert v is not None and v["validation_results"] == [{"v": 1}]
249         cands = db2.query_candidate_segments("vidA")
250         assert len(cands) == 1 and cands[0]["stats"]["net_crossings"] == 3
251
252
253     def test_migration_idempotent_on_interrupted_upgrade(tmp_path):
254         """An INTERRUPTED v3/v4 upgrade must not brick the DB. A DDL ``ALTER`` auto-commits
255         on its own, so a crash after the ADD COLUMNs but before the ``PRAGMA user_version``
256         bump leaves the columns present while the version is stale ? re-opening re-runs the
257         bare ALTERs. The ladder must tolerate that (re-runnable, like the v1/v2/v5 blocks)
258         instead of raising ``OperationalError: duplicate column name``."""
259         db_path = tmp_path / "interrupted.db"
260         Database(str(db_path))  # fully migrate to the current version (v3/v4 columns all
present)
261
262         # Simulate the interruption: the v3/v4 ADD COLUMNs already landed and auto-
committed,
263         # but the process died before the version bump, so user_version is stuck at 2.
264         conn = sqlite3.connect(str(db_path))
265         conn.execute("DROP TABLE IF EXISTS user_models")          # a true v2 DB lacks the
v5 table
266         conn.execute("PRAGMA user_version = 2")
267         conn.commit()
268         conn.close()
269
270         # Re-opening re-runs v3 (jobs.policy/next_poll_at), v4
(videos/candidate_segments.user_id),
271         # v5 (user_models), and v6 (jobs cost columns + pricebook) over artifacts that
ALREADY exist ?
272         # must NOT raise (re-runnable ALTERs / INSERT OR IGNORE), and must reach v6.
273         Database(str(db_path))
274         conn = sqlite3.connect(str(db_path))
275         assert conn.execute("PRAGMA user_version").fetchone()[0] == 6
276         conn.close()
277         # Columns/tables are all present after the clean re-apply.
278         assert "policy" in _table_cols(db_path, "jobs")
279         assert "cost_usd" in _table_cols(db_path, "jobs")  # v6 re-applied cleanly
280         assert "user_id" in _table_cols(db_path, "videos")
281         assert "user_id" in _table_cols(db_path, "candidate_segments")
282         conn = sqlite3.connect(str(db_path))
283         assert "user_models" in {r[0] for r in conn.execute(
284             "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
285         conn.close()
286
287
288     def test_add_column_idempotent_swallows_only_duplicate(tmp_path):
289         """The idempotent ADD COLUMN helper swallows ONLY 'duplicate column name'. Any other
290         OperationalError (e.g. a genuinely broken migration against a missing table) must
still
291         propagate ? the swallow is narrow, not a blanket except."""
292         db_path = tmp_path / "helper.db"
293         Database(str(db_path))                      # current schema; jobs.policy already
exists
294         conn = sqlite3.connect(str(db_path))
295         cur = conn.cursor()
296         # Re-adding an existing column is tolerated (the crash-safety path).
297         Database._add_column_idempotent(cur, "ALTER TABLE jobs ADD COLUMN policy TEXT")
298         # An unrelated OperationalError (no such table) is NOT masked.
```

```
        with pytest.raises(sqlite3.OperationalError):
            Database._add_column_idempotent(cur, "ALTER TABLE nope ADD COLUMN x TEXT")
        conn.close()


    # --- v5: per-user model store CRUD ------------------------------------------------

    def test_user_models_schema_contract(tmp_path):
        db_path = tmp_path / "test.db"
        Database(str(db_path))
        cols = _table_cols(db_path, "user_models")
        assert cols == {
            "id", "user_id", "version", "baseline_version", "feature_schema_hash",
            "fusion_config", "classifier_json", "promotion_evidence",
            "n_videos_at_fit", "is_active", "created_at",
        }


    def test_upsert_and_get_active_user_model_round_trip(tmp_path):
        db = Database(str(tmp_path / "test.db"))
        rid = db.upsert_user_model(
            "user-1", version=1, baseline_version="base-2026-06",
            feature_schema_hash="abc123",
            fusion_config={"min_crossings": 3, "cue_flags": {"person": True}},
            classifier_json={"model": "logreg", "w": [0.1, 0.2], "b": 0.0},
            promotion_evidence={"promote": True, "n": 6},
            n_videos_at_fit=6, is_active=True,
        )
        assert rid is not None

        m = db.get_active_user_model("user-1")
        assert m is not None
        assert m["user_id"] == "user-1"
        assert m["version"] == 1
        assert m["baseline_version"] == "base-2026-06"
```

17. user (2026-06-21T11:11:04.106Z)

```
      """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).

      Pure cost math + the v6 migration (cost columns + versioned pricebook) +
record/get/aggregate +
      the default-OFF wiring. Cardinal invariant: with the seeded ``placeholder-v0`` pricebook
(every
      rate 0.0) every cost is 0, and with ``billing.enabled=False`` NOTHING records (cost
columns stay
      NULL = today).
      """
      from backend import billing
      from backend.config import AppConfig
      from backend.infrastructure.database import Database


      # ------------------------------------------------------------------ pure cost module

      def test_compute_cost_zero_without_rates():
          cost, bd = billing.compute_cost_usd({"gpu_seconds": 180, "egress_gb": 0.04}, {})
          assert cost == 0.0 and bd == {"gpu_seconds": 0.0, "egress_gb": 0.0}


      def test_compute_cost_with_rates_and_gemini_passthrough():
          usage = {"gpu_seconds": 180, "egress_gb": 0.04, "gemini_usd": 0.05}
          rates = {"gpu_seconds": 0.0002, "egress_gb": 0.10}
          cost, bd = billing.compute_cost_usd(usage, rates)
          assert cost == round(180 * 0.0002 + 0.04 * 0.10 + 0.05, 6)  # 0.036 + 0.004 + 0.05
```

```
25          assert bd["gpu_seconds"] == 0.036 and bd["egress_gb"] == 0.004 and bd["gemini_usd"]
== 0.05
26
27
28      def test_apply_margin_clamps_and_to_inr():
29          assert billing.apply_margin(0.10, 0.20) == 0.12
30          assert billing.apply_margin(0.10, -5) == 0.10        # margin clamped to >= 0 (never
below cost)
31          assert billing.to_inr(1.0, 83.0) == 83.0
32
33
34      # ------------------------------------------------------------------ v6 migration +
pricebook seed
35
36      def test_v6_migration_adds_cost_columns_and_seeds_placeholder(tmp_path):
37          db = Database(str(tmp_path / "m.db"))
38          with db._connect() as conn:
39              ver = conn.execute("PRAGMA user_version").fetchone()[0]
40              cols = {r[1] for r in conn.execute("PRAGMA table_info(jobs)").fetchall()}
41              seeded = conn.execute(
42                  "SELECT unit_price_usd FROM pricebook WHERE
version='placeholder-v0'").fetchall()
43          assert ver >= 6
44          assert {"cost_usd", "gpu_active_seconds", "wall_seconds", "bytes_out", "owner_id",
45                  "pricebook_version", "cost_breakdown_json", "compute_backend", "gpu_type"}
<= cols
46          assert len(seeded) >= 4 and all(r[0] == 0.0 for r in seeded)  # placeholder = every
rate $0
47
48
49      def test_v6_migration_is_idempotent(tmp_path):
50          """Re-opening the DB re-runs the ladder guard (version already 6 ? no-op, no
duplicate seed)."""
51          path = str(tmp_path / "m.db")
52          Database(path)
53          db2 = Database(path)  # second open must not crash or duplicate the seed
54          with db2._connect() as conn:
55              n = conn.execute("SELECT COUNT(*) FROM pricebook WHERE
version='placeholder-v0'").fetchone()[0]
56          assert n == 4  # exactly the 4 seeded rows (INSERT OR IGNORE on the unique key)
57
58
59      # ------------------------------------------------------------------ pricebook rate
resolution
60
61      def _seed_real_pricebook(db, version="v1"):
62          with db._connect() as conn:
63              conn.executemany(
64                  "INSERT INTO pricebook (version, resource, gpu_type, unit_price_usd,
currency) "
65                  "VALUES (?, ?, ?, ?, 'USD')",
66                  [(version, "gpu_seconds", "L4", 0.0002, ), (version, "gpu_seconds", "T4",
0.0001, ),
67                   (version, "egress_gb", "", 0.10, ), (version, "wall_seconds", "", 0.0, )])
68              conn.commit()
69
70
71      def test_get_pricebook_rates_selects_by_gpu_type(tmp_path):
72          db = Database(str(tmp_path / "m.db"))
73          _seed_real_pricebook(db)
74          l4 = db.get_pricebook_rates("v1", "L4")
75          assert l4["gpu_seconds"] == 0.0002 and l4["egress_gb"] == 0.10
76          assert db.get_pricebook_rates("v1", "T4")["gpu_seconds"] == 0.0001
77          assert db.get_pricebook_rates("nonexistent", "L4") == {}  # unknown version ? no
rates ? cost 0
```

```
78
79
80      # ------------------------------------------------------------------ record / get /
aggregate
81
82      def test_record_and_get_job_cost(tmp_path):
83          db = Database(str(tmp_path / "m.db"))
84          db.create_job("j1", "v.mp4")
85          db.set_job_video_id("j1", "vidA")
86          _seed_real_pricebook(db)
87          out = db.record_job_cost("j1", usage={"gpu_seconds": 180, "egress_gb": 0.04,
"wall_seconds": 200},
88                                   pricebook_version="v1", gpu_type="L4",
compute_backend="cloud_run",
89                                   owner_id="u1")
90          assert out["cost_usd"] == round(180 * 0.0002 + 0.04 * 0.10, 6)
91          cost = db.get_job_cost("j1")
92          assert cost["cost_usd"] == out["cost_usd"]
93          assert cost["gpu_type"] == "L4" and cost["owner_id"] == "u1" and
cost["compute_backend"] == "cloud_run"
94          assert cost["bytes_out"] == int(0.04 * 1e9)
95          assert cost["cost_breakdown"]["gpu_seconds"] == 0.036
96
97
98      def test_record_with_placeholder_pricebook_is_zero(tmp_path):
99          db = Database(str(tmp_path / "m.db"))
100         db.create_job("j1", "v.mp4")
101         out = db.record_job_cost("j1", usage={"gpu_seconds": 180},
pricebook_version="placeholder-v0",
102                                  gpu_type="L4")
103         assert out["cost_usd"] == 0.0  # default placeholder ? $0, nothing user-facing
changes
104
105
106     def test_record_applies_margin(tmp_path):
107         db = Database(str(tmp_path / "m.db"))
108         db.create_job("j1", "v.mp4")
109         _seed_real_pricebook(db)
110         out = db.record_job_cost("j1", usage={"gpu_seconds": 100}, pricebook_version="v1",
gpu_type="L4",
111                                  margin=0.20)
112         assert out["cost_usd"] == round(100 * 0.0002 * 1.20, 6)
113
114
115     def test_get_job_cost_unknown_returns_none(tmp_path):
116         assert Database(str(tmp_path / "m.db")).get_job_cost("nope") is None
117
118
119     def test_get_video_cost_aggregates_across_jobs(tmp_path):
120         db = Database(str(tmp_path / "m.db"))
121         for jid in ("j1", "j2"):
122             db.create_job(jid, "v.mp4")
123             db.set_job_video_id(jid, "vidA")
124         _seed_real_pricebook(db)
125         db.record_job_cost("j1", usage={"gpu_seconds": 100}, pricebook_version="v1",
gpu_type="L4")
126         db.record_job_cost("j2", usage={"gpu_seconds": 50}, pricebook_version="v1",
gpu_type="L4")
127         agg = db.get_video_cost("vidA")
128         assert agg["total_cost_usd"] == round(100 * 0.0002 + 50 * 0.0002, 6)
129         assert len(agg["jobs"]) == 2
130
131
132     # ------------------------------------------------------------------ default-OFF wiring
133
```

```
134     def test_maybe_record_job_cost_noop_when_disabled(tmp_path, monkeypatch):
135         """DEFAULT-OFF: billing.enabled=False ? the hook records NOTHING (cost stays
NULL)."""
136         import backend.main as bmain
137         from backend.api import job_worker
138 
139         db = Database(str(tmp_path / "m.db"))
140         db.create_job("j1", "v.mp4")
141         monkeypatch.setattr(bmain, "db_instance", db, raising=False)
142         monkeypatch.setattr(bmain, "global_config", AppConfig(), raising=False)  #
billing.enabled=False
143         job_worker._maybe_record_job_cost({"id": "j1"}, {"usage": {"gpu_seconds": 180}})
144         assert db.get_job_cost("j1")["cost_usd"] is None  # nothing recorded
145 
146 
147     def test_maybe_record_job_cost_records_when_enabled(tmp_path, monkeypatch):
148         import backend.main as bmain
149         from backend.api import job_worker
150 
151         db = Database(str(tmp_path / "m.db"))
152         db.create_job("j1", "v.mp4")
153         monkeypatch.setattr(bmain, "db_instance", db, raising=False)
154         cfg = AppConfig.model_validate({"billing": {"enabled": True, "pricebook_version":
"placeholder-v0"}})
155         monkeypatch.setattr(bmain, "global_config", cfg, raising=False)
156         job_worker._maybe_record_job_cost({"id": "j1"}, {"usage": {"gpu_seconds": 180,
"gpu_type": "L4"}})
157         assert db.get_job_cost("j1")["cost_usd"] == 0.0  # recorded (0 with placeholder
pricebook)
158 
```

18. assistant (2026-06-21T11:11:09.226Z)

The test coverage is thorough. Now let me verify the circular import claim concretely and check whether `billing` importing `from __future__ import annotations` plus only `typing` truly has no backend imports. Let me also verify the migration's `_add_column_idempotent` is using `cursor` while the seed `INSERT OR IGNORE` and `CREATE TABLE` are on the same cursor (transaction semantics within the `with self._connect()` block).

Let me check one subtle point: in `_migrate_to_v6`, the `ALTER TABLE` statements auto-commit, but the `CREATE TABLE pricebook`, `CREATE UNIQUE INDEX`, and the seed `INSERT`s happen after. If the process is interrupted *between* the ALTERs and the pricebook seed, what happens on re-run? Let me trace this carefully.

19. user (2026-06-21T11:11:10.503Z)

```
292         def _migrate_to_v6(self, cursor):
293             # v6: per-job COST LEDGER (COMPUTE_DECOUPLED_SERVING M8, D8). NULLABLE cost
columns on
294             # `jobs` (NULL = no cost recorded = byte-identical to today; nothing records
until
295             # billing.enabled + a real pricebook) + a VERSIONED `pricebook` table. Cost is
stored in
296             # USD (matches GCP billing = one source of truth); INR is a display conversion
at a
297             # configurable FX rate. ALTER (not recreate) so v1-v5 job rows survive; ADD
COLUMNs are
298             # idempotent so an interrupted upgrade can re-run them. Additive ONLY ? no
served change.
299             for col, typ in (
300                 ("owner_id", "TEXT"),               # who pays (the non-owner user / tenant)
301                 ("compute_backend", "TEXT"),        # local_gpu | cloud_run | ...
302                 ("gpu_type", "TEXT"),               # L4 | T4 | ... (selects the per-second
GPU rate)
303                 ("gpu_active_seconds", "REAL"),
```

```
304                 ("wall_seconds", "REAL"),
305                 ("bytes_out", "INTEGER"),           # egress
306                 ("gemini_cost_usd", "REAL"),        # provider-reported, already USD
307                 ("pricebook_version", "TEXT"),
308                 ("cost_usd", "REAL"),               # computed total (USD = source of truth)
309                 ("cost_breakdown_json", "TEXT"),    # {gpu_seconds: .., egress_gb: ..,
gemini_usd: ..}
310             ):
311                 self._add_column_idempotent(cursor, f"ALTER TABLE jobs ADD COLUMN {col}
{typ}")
312             # Partial index keeps the NULL=no-cost fast path churn-free while making per-
owner billing cheap.
313             cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_owner ON jobs(owner_id) "
314                            "WHERE owner_id IS NOT NULL")
315             # Versioned pricebook: one row per (version, resource, gpu_type). Rates change +
differ by
316             # gpu_type ? re-version (never mutate a shipped version, so a recorded cost
stays auditable).
317             cursor.execute("""
318                 CREATE TABLE IF NOT EXISTS pricebook (
319                     version TEXT NOT NULL,
320                     resource TEXT NOT NULL,
321                     gpu_type TEXT NOT NULL DEFAULT '',
322                     unit_price_usd REAL NOT NULL DEFAULT 0.0,
323                     currency TEXT NOT NULL DEFAULT 'USD',
324                     effective_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
325                 )
326             """)
327             cursor.execute("CREATE UNIQUE INDEX IF NOT EXISTS idx_pricebook_key "
328                            "ON pricebook(version, resource, gpu_type)")
329             # Seed a PLACEHOLDER version with every rate 0.0 ? cost_usd computes to 0
(nothing
330             # user-facing changes). The owner INSERTs a real, calibrated version (real GCP
$/GPU-s,
331             # egress $/GB) and flips billing.pricebook_version to it ? never overwrite this
seed.
332             for resource, gpu_type in (("gpu_seconds", "L4"), ("gpu_seconds", "T4"),
333                                        ("wall_seconds", ""), ("egress_gb", "")):
334                 cursor.execute(
335                     "INSERT OR IGNORE INTO pricebook (version, resource, gpu_type,
unit_price_usd, "
336                     "currency) VALUES ('placeholder-v0', ?, ?, 0.0, 'USD')", (resource,
gpu_type))
337             cursor.execute("PRAGMA user_version = 6")
338
339         def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
340             """Register or update a video row WITHOUT touching its child segments.
341
```

20. assistant (2026-06-21T11:11:14.057Z)

Now let me verify the circular import concern empirically and confirm `billing.py` has no backend imports. Let me also check that `record_job_cost` uses `_execute_write` which opens its own connection ? confirming `get_pricebook_rates` and the UPDATE don't share a transaction (which is fine for additive writes but worth noting).

21. user (2026-06-21T11:11:24.885Z)

IMPORTS OK

22. user (2026-06-21T11:11:24.987Z)

14:from __future__ import annotations

```
16:from typing import Dict, Mapping, Tuple
```

23. assistant (2026-06-21T11:11:28.019Z)

No circular import ? `billing.py` imports only `__future__` and `typing`, and `import backend.billing` inside `database.py` works. Confirmed clean.

Now let me run the relevant test suites to confirm green, and check the cost API tests in test_api_endpoints.py.

24. user (2026-06-21T11:11:28.974Z)

```
1       """TestClient coverage for the FastAPI seam (previously zero ? the compiler SyntaxError
2       that broke `import backend.main` shipped green because nothing imported it)."""
3       import os
4       import subprocess
5       import sys
6       from unittest.mock import patch, MagicMock
7
8       import pytest
9
10      pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly where absent
11      from fastapi.testclient import TestClient
12
13      import backend.main as m
14      from backend.infrastructure.database import Database
15      from backend.config.models import AppConfig
16      from backend.pipeline.validators.base import ValidationResult
17
18
19      class _InlineExecutor:
20          """submit() runs the fn synchronously ? async jobs complete before the response
21          is inspected (full lifecycle coverage lives in test_async_jobs.py)."""
22
23          def submit(self, fn, *args, **kwargs):
24              fn(*args, **kwargs)
25
26
27      @pytest.fixture
28      def env(tmp_path, monkeypatch):
29          db = Database(str(tmp_path / "t.db"))
30          cfg = AppConfig()
31          monkeypatch.setattr(m, "db_instance", db)
32          monkeypatch.setattr(m, "global_config", cfg)
33          monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
34          client = TestClient(m.app)
35          return client, db, tmp_path, monkeypatch
36
37
38      def _video(tmp_path, name="m.mp4"):
39          p = tmp_path / name
40          p.write_bytes(b"not-a-real-video-but-hashable")
```

25. user (2026-06-21T11:11:30.177Z)

```
551:# --- M8 cost-ledger endpoints (COMPUTE_DECOUPLED_SERVING M8, D8)
----------------------------
553:def test_job_cost_endpoint_null_before_recording(env):
556:    r = client.get("/api/jobs/j1/cost")
559:    assert body["cost_usd"] is None and body["cost_inr"] is None   # nothing recorded yet
563:def test_job_cost_endpoint_unknown_404(env):
565:    assert client.get("/api/jobs/nope/cost").status_code == 404
568:def test_job_cost_endpoint_after_recording_shows_inr(env):
575:    db.record_job_cost("j1", usage={"gpu_seconds": 100}, pricebook_version="v1",
```

```
gpu_type="L4")
576:    body = client.get("/api/jobs/j1/cost").json()
577:    assert body["cost_usd"] == 0.1
578:    assert body["cost_inr"] == round(0.1 * 83.0, 2)   # USD?INR at the configured FX
581:def test_video_cost_endpoint_aggregates(env):
590:    db.record_job_cost("j1", usage={"gpu_seconds": 100}, pricebook_version="v1",
gpu_type="L4")
591:    db.record_job_cost("j2", usage={"gpu_seconds": 50}, pricebook_version="v1",
gpu_type="L4")
592:    body = client.get("/api/videos/vidA/cost").json()
593:    assert body["total_cost_usd"] == 0.15 and body["total_cost_inr"] == round(0.15 * 83.0,
2)
```

26. assistant (2026-06-21T11:11:33.847Z)

Let me check the route shadowing test concern more carefully ? there's a subtle issue. The endpoint `/api/jobs/{job_id}/cost` vs `/api/jobs/{job_id}`. But what about `/api/videos/{video_id}/cost`? Is there an existing `/api/videos/{video_id}` route that it could conflict with, or a route that `/api/videos/{video_id}/cost` could shadow? Let me check. Also let me run the test suites.

27. user (2026-06-21T11:11:35.373Z)

```
326:@app.get("/api/videos/{video_id}/cost")
```

28. user (2026-06-21T11:11:48.260Z)

```
.................................................................    [100%]
69 passed in 9.99s
```

29. assistant (2026-06-21T11:11:51.652Z)

`/api/videos/{video_id}/cost` is the only `/api/videos` route, so no shadowing there. Let me run the API endpoint tests too and confirm the full relevant set is green.

30. user (2026-06-21T11:12:05.627Z)

```
.............................                                        [100%]
29 passed in 11.13s
```

31. assistant (2026-06-21T11:12:14.578Z)

All green. Now let me examine a few subtle migration-safety edge cases the lens calls for.

1. **Interrupted upgrade where columns/pricebook are partially present but version stale.** The ALTERs auto-commit individually. The `CREATE TABLE pricebook` and `CREATE UNIQUE INDEX` use `IF NOT EXISTS`. The seed uses `INSERT OR IGNORE` on the unique key. So a re-run is clean. But there's one subtlety: the `CREATE UNIQUE INDEX idx_pricebook_key` is created on `(version, resource, gpu_type)`. The seed loop relies on `INSERT OR IGNORE` honoring that unique index. If for some reason the index creation failed but the table was created, duplicate seeds could appear. But within a single migration both run sequentially before commit, so this is consistent.

2. **A real edge case worth checking:** What if a user upgraded on an *older* version of this code path where `_migrate_to_v6` ran partially ? say the ALTERs landed and auto-committed, but `CREATE TABLE pricebook` failed/interrupted, leaving `user_version` still < 6. On re-run, the ALTERs are swallowed (duplicate), the CREATE IF NOT EXISTS runs, and seeds insert. Clean. Confirmed by `test_migration_idempotent_on_interrupted_upgrade`.

Let me verify one potential concern: the `get_video_cost` query selects `WHERE video_id = ?` but for a v0-v5 DB the `video_id` column on `jobs` ? does it exist? Let me confirm `jobs.video_id` predates v6.

32. user (2026-06-21T11:12:15.959Z)

```
63:        """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.
126:                video_id TEXT NOT NULL,
133:                FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
157:                video_id TEXT NOT NULL,
170:                FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
176:                ON candidate_segments(video_id, detector)
188:            CREATE TABLE IF NOT EXISTS jobs (
191:                video_id TEXT,
213:        # ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run them.
214:        self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
215:        self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
TIMESTAMP")
225:        # the ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run
them.
227:        self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id TEXT")
228:        self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
user_id TEXT")
287:    def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
302:            (video_id, filepath, status, json.dumps(validation_results)),
306:    def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
314:            p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE video_id =
?",
315:                            (video_id,)).fetchone()[0]
316:            c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
317:                            (video_id,)).fetchone()[0]
320:    def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
332:                cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
334:                cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
336:                cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <= ?",
(video_id, cand_upto))
338:                cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id > ?",
(video_id, cand_after))
341:    def add_play_segment(self, video_id: str, start_time: float, end_time: float, server:
Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]] = None)
-> Optional[int]:
346:                    "INSERT INTO play_segments (video_id, start_time, end_time, duration,
server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
347:                    (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
362:    def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
```

33. assistant (2026-06-21T11:12:22.485Z)

`jobs.video_id` exists in the base `jobs` table (line 191), so `get_video_cost`'s query is safe on all versions.

Now let me examine the one area that's a genuine candidate concern through this lens: the `_maybe_record_job_cost` reads `job.get("owner_id")`. The `job` dict is the claimed row. Does the claimed job row carry `owner_id`? On a **stale DB upgraded to v6**, the column exists but would be NULL for old jobs ? fine. But more importantly: is `owner_id` ever actually populated on the job at claim time? That's a functional completeness concern, not a safety one ? it would just store NULL. Within the default-OFF safety lens, that's acceptable.

Let me verify one real concern about the served-path / default-OFF guarantee: does `get_job` (the existing `/api/jobs/{job_id}` endpoint) now return the new cost columns, potentially changing the served response shape even when billing is OFF? Let me check `get_job`.

34. user (2026-06-21T11:12:24.814Z)

```
572:    def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
```

```
590:    def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
593:            row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
666:                "SELECT id, cost_usd, pricebook_version, cost_breakdown_json FROM jobs "
727:                    "SELECT id, status FROM jobs WHERE status='queued' "
```

35. assistant (2026-06-21T11:12:30.060Z)

Found a real, grounded issue. `get_job` at line 593 uses `SELECT * FROM jobs`, which means after the v6 migration adds 10 new columns, the serialized job dict returned by `GET /api/jobs/{job_id}` now includes `owner_id, compute_backend, gpu_type, gpu_active_seconds, wall_seconds, bytes_out, gemini_cost_usd, pricebook_version, cost_usd, cost_breakdown_json` ? all NULL when billing is off. Let me read `get_job` fully to confirm what's returned to the served path.

36. user (2026-06-21T11:12:30.703Z)

```
590         def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
591             """Fetch one job; `result`/`player_pool` JSON deserialized."""
592             with self._connect() as conn:
593                 row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
594                 if not row:
595                     return None
596                 d = dict(row)
597                 d["result"] = json.loads(d["result"]) if d["result"] else None
598                 d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
None
```

37. assistant (2026-06-21T11:12:36.237Z)

Confirmed. `get_job` does `SELECT *` and `dict(row)`, so the `GET /api/jobs/{job_id}` response now carries 10 extra NULL keys whenever the DB is at v6 ? independent of `billing.enabled`. This is a real deviation from the "served-path shape unchanged when billing is OFF" contract, though it's an additive (NULL-valued) change rather than a behavioral one. Let me check whether the existing `get_job` API test asserts an exact response shape (which would have caught this) or is lenient.

38. user (2026-06-21T11:12:37.877Z)

```
76:    job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
91:    job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
134:    job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
165:    job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
313:def test_parity_job_worker_vs_direct_cli_path(env, tmp_path, monkeypatch):
325:    jobj = client.get(f"/api/jobs/{rj.json()['job_id']}").json()
328:    jobd = client.get(f"/api/jobs/{rd.json()['job_id']}").json()
335:    jobf = client.get(f"/api/jobs/{rf.json()['job_id']}").json()
391:    job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
425:    job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
553:def test_job_cost_endpoint_null_before_recording(env):
556:    r = client.get("/api/jobs/j1/cost")
563:def test_job_cost_endpoint_unknown_404(env):
565:    assert client.get("/api/jobs/nope/cost").status_code == 404
568:def test_job_cost_endpoint_after_recording_shows_inr(env):
576:    body = client.get("/api/jobs/j1/cost").json()
```

39. assistant (2026-06-21T11:12:44.189Z)

The existing job tests access specific keys (lenient), so they don't catch the added NULL keys ? meaning the shape change ships silently. This confirms the finding: the contract claim "no served-path query reads them / cost columns stay NULL = today" is true for *values* but the `GET /api/jobs/{id}` *response shape* does gain 10 NULL keys post-migration. Whether that matters depends on consumers; it's a minor contract deviation worth flagging, not a blocker.

Now let me double-check the remaining lens items are clean. Let me verify there are no other places pinning version 5 (e.g., constants, conftest, other test files) that were missed.

40. user (2026-06-21T11:12:45.598Z)

```
tests\test_calibrate_local.py:21:    assert len(cl.local_grid()) == 5 * 3 * 3 * 4    # velocity
× merge × min_dur × min_crossings
tests\test_calibration.py:32:    assert len(splits) == 5
tests\test_config.py:24:    assert cfg.indexing.ai_max_workers == 5
tests\test_config.py:259:        assert cfg.indexing.ai_max_workers == 5
backend\infrastructure\database.py:105:            if version < 5:
backend\infrastructure\database.py:290:        cursor.execute("PRAGMA user_version = 5")
backend\features\rally_seq.py:9:MUST bump FEATURE_SCHEMA_VERSION; artifact manifests record the
version they were
backend\features\rally_seq.py:21:FEATURE_SCHEMA_VERSION = 1
backend\features\rally_seq.py:31:    "version": FEATURE_SCHEMA_VERSION,
tests\test_fusion_golden_telemetry.py:93:    assert starts[0]["gpu"]["temp_c"] == 70.0 and
starts[0]["sys"]["cpu_pct"] == 50.0
tests\test_fusion_hybrid.py:164:    assert extras_t["queue_length"] == 5
tests\test_gemini.py:48:    assert res[0]["metadata"]["shot_count"] == 5
tests\test_import_smoke.py:122:        "assert rs.FEATURE_SCHEMA['version'] ==
rs.FEATURE_SCHEMA_VERSION\n"
backend\eval\rally_seq_proto.py:46:    FEAT_NAMES, FEATURE_SCHEMA, FEATURE_SCHEMA_VERSION,
_spread,
tests\test_rally_seg_eval.py:170:    assert strat["long_tau"] == 5.0
tests\test_reporting_harvest.py:69:    assert env.video_meta.fps == 59.94
tests\test_run_telemetry.py:87:    assert rec["gpu"]["temp_c"] == 85.0 and rec["sys"]["cpu_pct"]
== 50.0
tests\test_storage.py:68:    assert be.stat(parse_uri(str(tmp_path / "a.txt"))).size == 5
tests\test_storage.py:169:    assert be.stat(ref).size == 5
tests\test_storage.py:260:    assert be.stat(ref).size == 5
tests\test_validators_base.py:44:    assert context["ffprobe_meta"]["fps"] == 59.94
tests\test_training_gen0.py:95:    from backend.features.rally_seq import FEAT_NAMES,
FEATURE_SCHEMA_VERSION
tests\test_training_gen0.py:97:    assert payload["feature_schema_version"] ==
FEATURE_SCHEMA_VERSION
tests\test_wasb_cache.py:73:    assert windows_done == 5
tests\test_windowing.py:102:    assert p.blob_fallback is True and p.blob_factor == 5.0
```

41. assistant (2026-06-21T11:13:00.593Z)

No remaining schema-version pins reference `== 5` other than the migration ladder itself (`version < 5` and `PRAGMA user_version = 5` inside `_migrate_to_v5`, which are correct). All test assertions for `user_version` are updated to `6`. The version bump is fully propagated.

Let me do a final check on whether `record_job_cost`'s `_execute_write` opens a separate connection from `get_pricebook_rates` ? confirming this is just a non-atomic two-step (read rates, then write), which is fine for an additive single-row update but means there's no transactional coupling. This is not a safety issue for default-OFF.

I have enough to report. Let me compile the findings through the MIGRATION & DEFAULT-OFF SAFETY lens.

Summary of lens checks:
1. `_migrate_to_v6` additive-NULL + idempotent: YES ? re-runnable ALTERs via `_add_column_idempotent`, `CREATE ... IF NOT EXISTS`, `INSERT OR IGNORE` on the unique key. Interrupted-upgrade re-run is clean and covered by `test_migration_idempotent_on_interrupted_upgrade`.
2. `billing.enabled=False` behavior: `_maybe_record_job_cost` is a strict no-op (early return) and best-effort (try/except, logs warning, never raises into the job). Cost *values* stay NULL. **However**, the `GET /api/jobs/{id}` served response *shape* gains 10 NULL keys post-v6 because `get_job` does `SELECT *` ? a minor contract deviation.
3. v5?v6 version pins: all updated across the suite. Clean.
4. Circular import: none ? `billing.py` imports only `__future__`/`typing`; verified empirically.

5. Route ordering: `/api/jobs/{id}/cost` registered before `/api/jobs/{id}`; no shadowing. `/api/videos/{id}/cost` is the only `/api/videos` route.

One additional grounded observation: `to_inr(None, fx)` would crash, but `get_job_cost` endpoint guards it with `if usd is not None`. The `get_video_cost` endpoint passes `total_cost_usd` which is always a float (defaults 0.0), so safe. No issue there.

42. user (2026-06-21T11:13:09.554Z)

Structured output provided successfully

43. assistant (2026-06-21T11:13:21.840Z)

I completed the MIGRATION & DEFAULT-OFF SAFETY review of the M8 cost-ledger change on `feat/serving-m8-cost-ledger`.

Verdict: the migration and default-OFF design are sound. I verified all five lens questions and found one minor, real issue.

What checked out clean:
1. **Additive-NULL + idempotent migration** ? `_migrate_to_v6` (database.py:292) uses re-runnable `_add_column_idempotent` ALTERs, `CREATE ... IF NOT EXISTS`, and `INSERT OR IGNORE` on the unique pricebook key. The interrupted-upgrade path (columns present, version stale) re-runs cleanly, covered by `test_migration_idempotent_on_interrupted_upgrade` (test_database.py:253).
2. **Default-OFF wiring** ? `_maybe_record_job_cost` (job_worker.py) is a strict early-return no-op when `billing.enabled=False` and best-effort (try/except, never raises into the job).
3. **v5?v6 version pins** ? all updated to `6` across test_database.py, test_async_jobs.py, test_job_worker_loop.py, test_billing.py; no stale `== 5` pins remain.
4. **No circular import** ? `billing.py` imports only `__future__`/`typing`; verified empirically that `database`/`billing`/`main` all import.
5. **No route shadowing** ? `/api/jobs/{id}/cost` (main.py:311) is registered before `/api/jobs/{id}` (main.py:337); `/api/videos/{id}/cost` is the only `/api/videos` route.

The relevant suites are green locally (69 passed across billing/database/async-jobs/job-worker; 29 passed in test_api_endpoints.py).

The one finding (minor): `get_job` at `backend/infrastructure/database.py:593` uses `SELECT *`, so once v6 lands, the `GET /api/jobs/{id}` response gains 10 NULL cost keys for every job ? and this is keyed off the *migration*, not the billing flag, so it persists even when billing is OFF. It is additive-NULL (no behavioral change), but it is a real deviation from the stated "served path unchanged when billing is OFF" contract, and the existing key-lenient job tests let it ship silently. Details and fix options are in the structured finding.